PW EARNERS

DSA in C++

DPP — 10 (SOLUTIONS)

Topic: Recursion

Multiple Choice Questions (Predict the Output & Theory)

Q1. Recursive Summation
Correct Option: **B (15)**
Explanation: The function calculates the sum of the first n natural numbers. `f(5)` calls `f(4) + 5`, which calls `f(3) + 4`, and so on until `f(1)` returns 1. The total sum is 1+2+3+4+5 = 15.

.....

Q2. Time Complexity Analysis
Correct Option: **C (O(N log N))**
Explanation: The function does $O(N)$ work in the `for` loop and makes 2 recursive calls of size $n/2$. The recurrence relation is $T(N) = 2T(N/2) + O(N)$. According to the Master Theorem, this results in a time complexity of $O(N \log N)$, exactly like the Merge Sort algorithm.

.....

Q3. Pass by Reference in Recursion
Correct Option: **B (11)**
Explanation: Since `x` is passed by reference, all changes directly affect the original variable `p`. The recursive function runs 3 times (for `c = 3, 2, 1`). Each time it adds 2 to `x`. So, p becomes $5 + 2 + 2 + 2 = 11$.

.....

Q4. Multiple Function Recursion
Correct Option: **A (5 4 2 2 5)**
Explanation: Let's trace the calls:
1. `funA(5)` prints 5, calls `funB(4)`, and awaits to print 5.
2. `funB(4)` prints 4, calls `funA(2)`.
3. `funA(2)` prints 2, calls `funB(1)`, and awaits to print 2.
4. `funB(1)` hits the base case (`n <= 1`) and returns.
Backtracking resolves the remaining `cout` statements in `funA`, printing the delayed 2 and 5. Output: `5 4 2 2 5`.

.....

Q5. Tree Recursion
Correct Option: **A (3 2 1 1 2 3)**
Explanation: The function prints the number, recursively calls for `n - 1`, and then prints the number again during the returning phase. `printNum(3)` prints 3, calls `printNum(2)`, then prints 3. `printNum(2)` prints 2, calls `printNum(1)`, prints 2. `printNum(1)` prints 1, calls `printNum(0)`, prints 1. Combining them sequentially gives `3 2 1 1 2 3`.

.....

Q6. Nested Recursion
Correct Option: **C (91)**
Explanation: This is a famous recursion sequence known as McCarthy 91 function. For any argument $n \leq 100$, the function rigorously evaluates through nested calls to always return 91.

.....

Q7. Edge Case Challenge
Correct Option: **C (Results in a Stack Overflow)**
Explanation: `crashCourse(n-)` uses the post-decrement operator. This means it passes the *current* value of `n` to the recursive call, and *then* decreases the local `n`. Thus, `crashCourse(3)` infinitely calls `crashCourse(3)` without shrinking the input, resulting in a Stack Overflow.

.....

Q8. Complete the Code (Binary Search)
Correct Option: **A**
Explanation:
1: `low > high` (Base case: element not found).
2: `low + (high - low)/2` (Avoids integer overflow).
3: `mid` (Return index if found).
4: `binarySearch(arr, low, mid - 1, target)` (Search left half).
5: `binarySearch(arr, mid + 1, high, target)` (Search right half).

.....

Q9. Reverse a String

```cpp
#include <iostream>
#include <string>
#include <utility>
using namespace std;

void reverseString(string &s, int left, int right) {
    // Base case: If left pointer crosses or reaches right, stop
    if (left >= right) return;

    // Swap the characters at boundaries
    swap(s[left], s[right]);

    // Recursive call for inner string
    reverseString(s, left + 1, right - 1);
}

int main() {
    string s;
    cin >> s;
    reverseString(s, 0, s.length() - 1);
    cout << s << endl;
}
```

Explanation: We use two pointers, `left` at the beginning and `right` at the end. We swap the characters at these indices and recursively solve for the remaining substring by passing `left + 1` and `right - 1`.

Q10. Valid Palindrome

```cpp
#include <iostream>
#include <string>
using namespace std;

bool isPalindrome(string &s, int left, int right) {
    // Base Case: If pointers cross, all characters matched!
    if (left >= right) return true;

    // If mismatch found, it's not a palindrome
    if (s[left] != s[right]) return false;

    // Check for the inner substring
    return isPalindrome(s, left + 1, right - 1);
}

int main() {
    string s;
    cin >> s;
    if (isPalindrome(s, 0, s.length() - 1)) {
        cout << "true\n";
    } else {
        cout << "false\n";
    }
}
```

Explanation: Similar to string reversal, we compare the characters at `left` and `right`. If they don't match, we immediately return `false`. If they do, we recursively ask the function to check the substring inside.

Q11. Power of Three

```cpp
#include <iostream>
using namespace std;

bool isPowerOfThree(int n) {
    if (n <= 0) return false;       // 0 and negatives are not powers of 3
    if (n == 1) return true;        // 3^0 = 1

    if (n % 3 != 0) return false;   // If not divisible by 3, false

    // Shrink the problem
    return isPowerOfThree(n / 3);
}

int main() {
    int n;
    cin >> n;
    if (isPowerOfThree(n)) cout << "true\n";
    else cout << "false\n";
```

```
}
```

Explanation: A number is a power of 3 if dividing it repeatedly by 3 eventually leaves us with 1. If at any point the number is not perfectly divisible by 3, it cannot be a power of three.

.....

Q12. GCD of Two Numbers

```cpp
#include <iostream>
using namespace std;

// Euclidean Algorithm for GCD
int gcd(int a, int b) {
    if (b == 0) return a;
    return gcd(b, a % b);
}

int main() {
    int q;
    cin >> q;
    while(q--) {
        int a, b;
        cin >> a >> b;
        cout << gcd(a, b) << endl;
    }
}
```

Explanation: This code implements the highly efficient Euclidean algorithm. The GCD of `A` and `B` is the same as the GCD of `B` and `A % B`. This cuts down the input size logarithmically, ensuring the strict time complexity constraint is met.

.....

Q13. Check if Array is Sorted

```cpp
#include <iostream>
#include <vector>
using namespace std;

bool isSorted(vector<int>& arr, int idx) {
    // If we reach the last element, the array is sorted
    if (idx == arr.size() - 1) return true;

    // If the current element is greater than or equal to the next,
       it's not strictly increasing
    if (arr[idx] >= arr[idx + 1]) return false;

    // Recursively check the rest of the array
    return isSorted(arr, idx + 1);
}

int main() {
    int n;
    cin >> n;
    vector<int> arr(n);
```

```
    for(int i = 0; i < n; i++) {
        cin >> arr[i];
    }
    if (isSorted(arr, 0)) cout << "true\n";
    else cout << "false\n";
}
```

Explanation: We start from index 0. At any position `idx`, if `arr[idx] >= arr[idx + 1]`, the strictly increasing property is violated, and we return `false`. Otherwise, we recursively ask to verify from `idx + 1`.

.....

Q14. Decode String

```
#include <iostream>
#include <string>
using namespace std;

string decodeString(string& s, int& i) {
    string result = "";

    while (i < s.length() && s[i] != ']') {
        if (isdigit(s[i])) {
            int k = 0;
            // Build the full number (might be multiple digits)
            while (i < s.length() && isdigit(s[i])) {
                k = k * 10 + (s[i] - '0');
                i++;
            }

            i++; // Skip the '['
            string decodedString = decodeString(s, i);
            i++; // Skip the ']'

            // Append decoded string 'k' times
            while (k-- > 0) {
                result += decodedString;
            }
        } else {
            result += s[i];
            i++;
        }
    }
    return result;
}

int main() {
    string s;
    cin >> s;
    int index = 0;
    cout << decodeString(s, index) << endl;
}
```

Explanation: The key is passing the `index i` **by reference** so that recursive calls naturally

progress through the string without duplicating work. When we hit a number, we calculate it, skip `'['`, and invoke recursion to solve the inner brackets. When the nested call encounters `']'`, it returns, and we multiply the returned string `k` times.

.....

Q15. K-th Symbol in Grammar (Hard)

```cpp
#include <iostream>
using namespace std;

int kthGrammar(int n, int k) {
    // Base Case: First row is always 0
    if (n == 1) return 0;

    // Calculate the midpoint of the n-th row using bitwise left shift.
    // The n-th row has 2^(n-1) elements, so mid is 2^(n-2).
    // Using 1 << (n - 2) is a very fast way to calculate powers of 2.
    int mid = 1 << (n - 2);

    if (k <= mid) {
        // Left half is an exact replica of the previous row
        return kthGrammar(n - 1, k);
    } else {
        // Right half is the inverse of the previous row's corresponding element
        return 1 - kthGrammar(n - 1, k - mid);
    }
}

int main() {
    int n, k;
    cin >> n >> k;
    cout << kthGrammar(n, k) << endl;
}
```

Explanation: If you observe the pattern carefully:
Row 1: `0`
Row 2: `01`
Row 3: `0110`
Row 4: `01101001`

Notice that the **first half** of any row N is an exact copy of row $N-1$. The **second half** of row N is the exact complement (inverted version) of row $N-1$.

We can solve this recursively by finding the midpoint. For row N, length is 2^{N-1}. Midpoint is 2^{N-2}. We calculate this optimally using bitwise shift: `1 << (n - 2)`.

Dry Run for Sample 1 (`n = 4, k = 5`):

- `kthGrammar(4, 5)`: Midpoint for row 4 is $2^2 = 4$. Here, $k(5) > 4$. So we fall in the right half! It will be the inverse of the 1st element $(5-4)$ of the previous row.
- Return `1 - kthGrammar(3, 1)`.
- `kthGrammar(3, 1)`: Midpoint for row 3 is $2^1 = 2$. Here, $k(1) \leq 2$. We are in the left half, so

it perfectly matches the previous row's 1st element.

- Return `kthGrammar(2, 1)`.
- `kthGrammar(2, 1)`: Midpoint for row 2 is $2^0 = 1$. Here $k(1) \leq 1$.
- Return `kthGrammar(1, 1)`.
- `kthGrammar(1, 1)`: Base case reached! Returns `0`.

Now backtrack the values:
`kthGrammar(2, 1)` returns `0`.
`kthGrammar(3, 1)` returns `0`.
`kthGrammar(4, 5)` returns `1 - 0 = 1`. Output is `1`!

All the best! Keep Coding ♡